Designing and writing Script Extender mods

Guide covering the generation of ideas, design, and implementation of SE mods, providing practical tips throughout. Includes an example mod, 'Waypoint Inside Emerald Grove', to illustrate the process, offering insights into the tools and methods used.

Designing and writing Script Extender mods for Baldur's Gate III

written by Volitio

As an author that has created dozens of SE-only mods, I thought I'd share with the community some of my experiences. In this guide, I will outline my process for creating mods, while providing some tips for aspiring modders.

Now, just to make it clear:

- This guide assumes little programming and SE knowledge, which could mostly be obtained by following the basic tutorials present in this wiki. Even then, most of this applies to modding in general.
- I got into modding because designing and problem solving are terribly addicting, and the game is great.
- Modding is a creative process and should be fun, and there are many ways to go about it. This guide is just *my* way of doing things, and it's not to say other ways are wrong.

1. Idea

To me, the most important part of creating a mod is having a good idea. Apart from my frameworks such as ISF and MCM, I don't really have complex mods; I like to think I just have good ideas. I usually get my ideas from playing the game and thinking "I wish I could do *this*", "I wish *this* was different", "*This* could be automated". I know many modders don't even have reached Act 2, but I think it's important to play the game to get a feel for it and its game design. I also get ideas from other mods, or grievances within the modding community (sometimes, you'll notice these by just writing mods yourself). I usually write down my ideas somewhere and come back to them later.

Therefore, my first tip is to play the game and think about what you'd like to change. If you're a new modder, I'd recommend starting with

something simple, like a small tweak or a quality of life mod. You can always expand on your ideas later.

Don't get too caught up in the details or feasibility of your idea yet, you're not bound or committed to anything.

2. Design

Once you have a solid idea, the next step is to design your mod. This also involves planning how your mod will function and what changes it will implement in the game. By taking the time to design your mod thoughtfully, you can create a more polished and enjoyable experience for players, while also making the development process smoother for yourself. If you don't have a clear idea of what you want to achieve, you'll likely face more challenges and setbacks during development.

Good mod design involves considering questions such as:

- What is the purpose of the mod?
- Is it solving a problem? What problem should it solve? Why does that problem exist?
  - Can we solve the root cause of the problem?
  - What features will it include to solve that problem?
- How will it fit into the existing game mechanics?
- How will players interact with it? Can it be integrated into the game seamlessly?
- What will players expect from the mod when hearing about it or using it without prior knowledge?
- Are there potential edge-cases that need to be considered?
- Does it call for different options or settings? What would be sane defaults?

Features and scope

You should consider the scope of your mod, and how easy would it be to shrink or expand it. Being able to identify parts to trim and keeping the scope manageable will help you stay focused and avoid burnout.

Outline the specific features you want to include. Consider how these features will interact with the game and with each other.

Also, think about how players will interact with your mod. Aim for an intuitive design that enhances gameplay without feeling weird to users. Vanilla game design is a good reference point, and it's what players are used to. I personally always aim for a Vanilla+ experience with my mods.

3. Implementation

Once you have a clear design for your mod, you can start implementing it. This involves writing the code that will make your mod work. If you're new to modding, this can be a challenging step, but it's also where you'll learn the most. With time, you'll be more familiar with the modding capabilities of SE.

In this section, I'll cover some tips for learning to learn SE.

1. **Start small**: If you're new to modding, it's a good idea to start with a small project. This will help you get familiar with the tools and techniques involved in modding and the general workflow cycle, without getting overwhelmed.
2. **Read the documentation**: The SE documentation is a great resource; take the time to read through it and familiarize yourself with the different functions and features available. The documentation may be lacking in some areas, but it's a good starting point.
3. **Symlink, symlink, symlink**: Symlinks are your best friend when modding with SE! This allows you to test your mods without having to repak your mod and restart the game for every mundane change. This can save you a lot of time and frustration, especially when you're just starting out, and will accelerate your learning process a million times. ***==If there's one thing you should take away from this guide, it's to use symlinks!==***
4. **Experiment**: The best way to learn is by doing. Having symlinked your mod, experiment with different functions and entities. One great thing you can do is dumping entities during runtime at two different points in time, and comparing them. Sometimes you want to know if something changed after a certain action, and this is a great way to do it (apart from checking flags and such).
   You can use Scribe to speed up this process even further.

If you have:

- Correctly set up your environment (mod folder, symlinks, etc.)
- Reference files (SE IDE helpers, Osiris functions, events, PROCs, etc.)
- A clear mod design

You should be able to start iterating on your mod. Sometimes, things just can't be properly implemented, or some API is missing. Sometimes, you'll have to change your design or scope, and that's okay. It's part of the process.

If you don't know how to do something:

- Think if this is something that can be tackled with SE or Osiris. As you get more experienced, you'll be able to tell the difference and the trade-offs
  - For SE, you can check the documentation, the source code , or search & ask the Discord servers.
  - For Osiris, check if there are relevant functions or events.
- If you need to check for some state/progression, you can check the existing Flags dump, or dump entities and see what's there, doing a diff between two dumps (before and after) with VSCode. There are ways to do this with frame components too; again, check Scribe.

Modding is like a puzzle, and you'll have to find the right pieces to make it work. Sometimes, you'll have to make your own pieces, and to me that is the beauty and art of modding.

Example: 'Waypoint Inside Emerald Grove'

In this section, I will reproduce in prose what would be like designing a mod, using my mod 'Waypoint Inside Emerald Grove' as an example.

Idea

Whether you've only played the EA or has never reached the goblin camp, you've probably noticed that the Emerald Grove is a bit of a pain to navigate. The nearest waypoint is like 10 seconds from the gate, which takes more ~5 seconds to open, and then you have to walk all the way to the grove for a few more seconds. For every trip to get to Dammon, you'll likely spend tens of seconds just walking.
Notice that the idea presents itself: *having a waypoint inside the Emerald Grove*.

Design

The design should be pretty simple, right? Just add a waypoint inside the Emerald Grove. But there are some things to consider:

- Where should the waypoint be placed? The grove is still a large area, maybe it could be dynamically changed to provide the most convenient location for players…
- It would be really great if we could add it to the map, just like the other waypoints, to make it easily accessible.
- Being a waypoint, it must be accessible only after the player has reached the Grove. How do we manage that unlock condition? Should it be tied to a specific quest or event?

- Are there any lore implications to consider? Was it intended for the Grove not to have a waypoint, or could it simply be justified as a new addition for player convenience?
- What happens if the Grove enters lockdown? The waypoint should be disabled or removed to maintain consistency with the game's mechanics.
- How will the waypoint interact with existing game systems and features? Will it create any unintended consequences or balance issues?

Now we have a clearer idea of what the mod sets out to do, and some of the things we need to consider when implementing it. We'll need to manage the waypoint's state somehow, unlocking and locking it according to the game's state (flags). Lorewise, I couldn't find any reason for the Grove not to have a waypoint, so we're clear on that - even then, it's a mod meant to be convenient, and it does not break balance in any way.

Implementation

With the design in place, it's time to implement the mod. Funnily enough, this mod has gone through a few iterations, and there could be an additional one with the unlocked toolkit; however, since Larian has not publicly addressed the unlocked toolkit as of writing this guide, I'll describe the implementation without it.

Here's a breakdown of how I approached the implementation:

- Looked through Osiris and SE to gather things related to waypoints and flags.
  Osiris functions:

```
function Osi.UnlockWaypoint(waypointName, trigger, character) end
function Osi.OpenWaypointUI(character, currentWaypoint, item, isFleeing) end
function Osi.RegisterWaypoint(waypointName, item) end
function Osi.LockWaypoint(waypointName, character) end
```

  Osiris events:

```
function Osi.TeleportToFleeWaypoint(character, trigger) end
```

```
function Osi.TeleportToWaypoint(character,
trigger) end
```

From `ExtIdeHelpers` (SE-related), nothing too relevant:

```
--- @class PartyWaypoint
--- @field Level FixedString
--- @field Name FixedString
--- @field field_8 Guid

--- @class
PartyWaypointsComponent:BaseComponent
--- @field Waypoints Array_PartyWaypoint
```

With these things in mind, looks like `Osi.RegisterWaypoint`, `Osi.UnlockWaypoint` and `Osi.LockWaypoint` are the functions we need to use. We can do that right away!
Unfortunately, calling these functions without proper setup will not work. We need to set up the waypoint in XMLs first. In the first iteration of the mod, I didn't know this, so I piggybacked on an existing waypoint (which was not great, but worked).

For the piggybacking method, I experimented with the `Osi.TeleportToWaypoint` event, which gives you the waypoint name and the character that triggered it. I teleported to the Emerald Grove Environs waypoint, took note of the waypoint name, then used it to teleport the player to the Emerald Grove if that waypoint was used.

This way, whenever the player used the waypoint, they'd be teleported to the Emerald Grove, effectively creating a waypoint inside the grove. This also opens up the possibility to select where to teleport the player to, as opposed to traditional waypoints. I decided to offer three options: the grove entrance (Arron), next to Dammon, and next to the ritual (Sacred Pool).

This approach is a bit hacky and there's still a lot of ground to cover: the teleporting should only happen if the player has reached the grove, and until the grove gets locked down. Also, there should be a way to use the original waypoint location. Furthermore, just teleporting feels off, it would be great to have a visual effect when teleporting.

To solve this, I employed:

1. A flag to check if the player has reached the grove. You need to sift through the flags or Osiris logs to find the right one. It can be a bit

tedious.

2. A flag to check if the grove is locked down.
3. A way to allow players to use the original waypoint location. I decided to allow that with a config option to do so if the player is crouching. Dumping entities is a great way to find out if the player is crouching (sneaking/hiding).
4. A visual effect when teleporting. I used https://bg3.norbyte.dev/ to search for visual effects, and experimented with a few with `Osi.PlayEffect` and similar functions.

The coordinates to teleport to were obtained by using `Osi.GetPosition(Osi.GetHostCharacter())` in places where I wanted to teleport the player to.

From then on, it was a matter of setting up the code to handle these cases. The configuration options were handled manually with my config manager, but nowadays MCM is used.

Later on, I learned that waypoints could be set up in XMLs, and I eventually did that. This allowed me to have a proper waypoint inside the grove, having a separate option in the waypoint UI. I did however keep the teleporting logic, as it would allow having different destinations, since waypoints are static.

I'm confident that my other mods can offer similar insights and exercises into how to design and implement mods. I hope this guide has been helpful and inspiring to you, and I wish you the best of luck in your modding endeavors!
If you have any questions or need help, the BG3 Modding Community Discord server is a great place to ask for help.